Istio Service Mesh: Getting Started Lab Report

1. Overview

This lab demonstrates the deployment of the Bookinfo sample application on a Kubernetes cluster using Istio service mesh. The lab covers the installation of Istio, enabling Gateway API, deploying the Bookinfo application, exposing it via an ingress gateway, and accessing the observability dashboards.

2. Environment

Item	Specification
OS	Linux (Ubuntu 22.04)
Kubernetes	minikube v1.32.0
Istio	1.29.2
Container Runtime	Docker
Browser	Chrome

3. Step-by-Step Execution

3.1 Download and Install Istio

```
u6237@Yezo:~$ curl -L https://istio.io/downloadIstio | sh -
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   101  100   101    0     0     47      0  0:00:02  0:00:02 --:--:--    47
100  5124  100  5124    0     0    678      0  0:00:07  0:00:07 --:--:--  1146

Downloading istio-1.29.2 from https://github.com/istio/istio/releases/download/1.29.2/istio-1.29.2-linux-amd64.tar.gz ...

Istio 1.29.2 download complete!

The Istio release archive has been downloaded to the istio-1.29.2 directory.

To configure the istioctl client tool for your workstation,
add the /home/u6237/istio-1.29.2/bin directory to your environment path variable with:
         export PATH="$PATH:/home/u6237/istio-1.29.2/bin"

Begin the Istio pre-installation check by running:
         istioctl x precheck

Try Istio in ambient mode
        https://istio.io/latest/docs/ambient/getting-started/
Try Istio in sidecar mode
        https://istio.io/latest/docs/setup/getting-started/
Install guides for ambient mode
        https://istio.io/latest/docs/ambient/install/
Install guides for sidecar mode
        https://istio.io/latest/docs/setup/install/

Need more information? Visit https://istio.io/latest/docs/
u6237@Yezo:~$ cd istio-1.29.2
u6237@Yezo:~/istio-1.29.2$ export PATH=$PWD/bin:$PATH
```

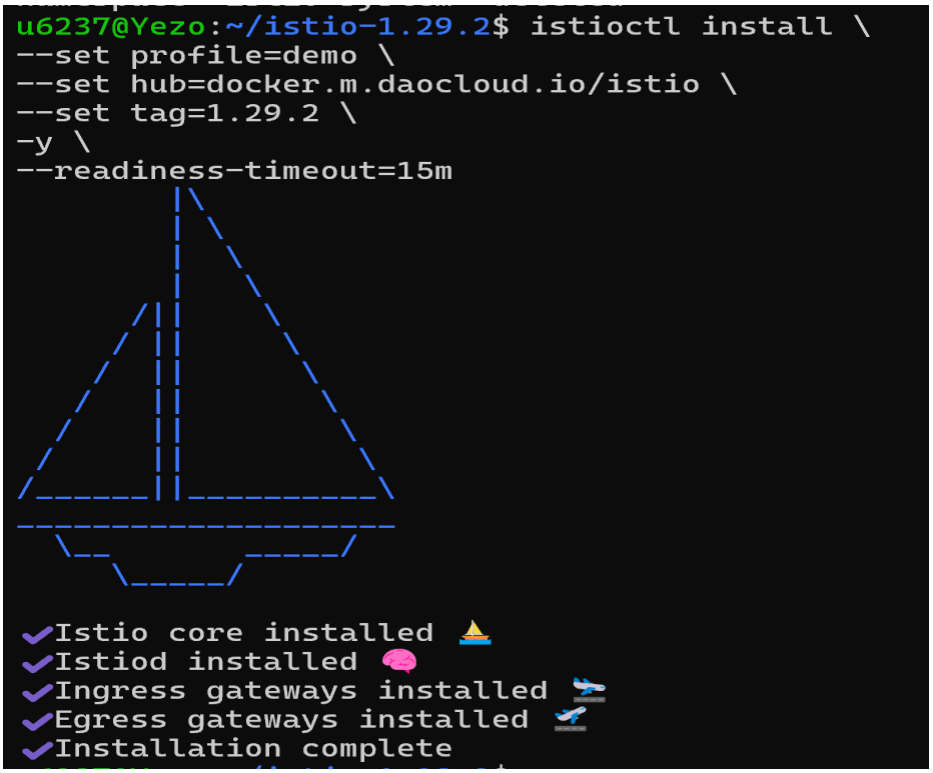
3.2 Install Istio with Gateway API Support

```
u6237@Yezo:~/istio-1.29.2$ istioctl install -f samples/bookinfo/demo-profile-no-gateways.yaml -y
        |\
        | \
        |  \
        |   \
      /||    \
     / ||     \
    /  ||      \
   /   ||       \
  /    ||        \
 /     ||         \
/______||__________\
____________________
  \__       _____/
     \_____/

✔Istio core installed ⛵
✗ Istiod encountered an error: failed to wait for resource: resources not ready after 5m0s: context deadline exceeded
  Deployment/istio-system/istiod (container failed to start: ImagePullBackOff: Back-off pulling image "docker.io/istio/p
ilot:1.29.2": ErrImagePull: Error response from daemon: Get "https://registry-1.docker.io/v2/": context deadline exceede
d)
Error: failed to install manifests: failed to wait for resource: resources not ready after 5m0s: context deadline exceed
ed
  Deployment/istio-system/istiod (container failed to start: ImagePullBackOff: Back-off pulling image "docker.io/istio/p
ilot:1.29.2": ErrImagePull: Error response from daemon: Get "https://registry-1.docker.io/v2/": context deadline exceede
d)
```

Here we encountered a problem: pulling the image link always timed out. The schematic diagram after troubleshooting is shown below:

```
u6237@Yezo:~/istio-1.29.2$ istioctl install \
--set profile=demo \
--set hub=docker.m.daocloud.io/istio \
--set tag=1.29.2 \
-y \
--readiness-timeout=15m
```



3.3 Label Namespace for Automatic Sidecar Injection

```
u6237@Yezo:~/istio-1.29.2$ kubectl label namespace default istio-injection=enabled
namespace/default labeled
```

3.4 Deploy Bookinfo Application

```
u6237@Yezo:~/istio-1.29.2$ kubectl get crd gateways.gateway.networking.k8s.io &> /dev/null || \
>
> { kubectl kustomize "github.com/kubernetes-sigs/gateway-api/config/crd?ref=v1.4.0" | kubectl apply -f -; }
customresourcedefinition.apiextensions.k8s.io/backendtlspolicies.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/gatewayclasses.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/gateways.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/grpcroutes.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/httproutes.gateway.networking.k8s.io created
customresourcedefinition.apiextensions.k8s.io/referencegrants.gateway.networking.k8s.io created
```

```
u6237@Yezo:~/istio-1.29.2$ kubectl apply -f samples/bookinfo/platform/kube/bookinfo.yaml
service/details created
serviceaccount/bookinfo-details created
deployment.apps/details-v1 created
service/ratings created
serviceaccount/bookinfo-ratings created
deployment.apps/ratings-v1 created
service/reviews created
serviceaccount/bookinfo-reviews created
deployment.apps/reviews-v1 created
deployment.apps/reviews-v2 created
deployment.apps/reviews-v3 created
service/productpage created
serviceaccount/bookinfo-productpage created
deployment.apps/productpage-v1 created
```

```
u6237@Yezo:~/istio-1.29.2$ kubectl get services
NAME          TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
details       ClusterIP   10.107.17.125    <none>        9080/TCP   26s
kubernetes    ClusterIP   10.96.0.1        <none>        443/TCP    14d
productpage   ClusterIP   10.111.224.251   <none>        9080/TCP   26s
ratings       ClusterIP   10.103.196.64    <none>        9080/TCP   26s
reviews       ClusterIP   10.107.32.239    <none>        9080/TCP   26s
```

Verify all pods are running:

```
u6237@Yezo:~/istio-1.29.2$ kubectl get pods
NAME                              READY   STATUS    RESTARTS   AGE
details-v1-86bc85c44b-8ccgp       2/2     Running   0          3m21s
productpage-v1-7c6b4d69c7-rfd4w   2/2     Running   0          3m21s
ratings-v1-6c55856598-nbj6f       2/2     Running   0          3m20s
reviews-v1-fff59d896-x6tvr        2/2     Running   0          3m20s
reviews-v2-bff87fc54-ztzxs        2/2     Running   0          3m19s
reviews-v3-59987ddb8d-hts6b       2/2     Running   0          3m19s
```

Note: Each pod has **2/2 containers** (application container + Istio sidecar proxy)

3.5 Verify Bookinfo Application Internally

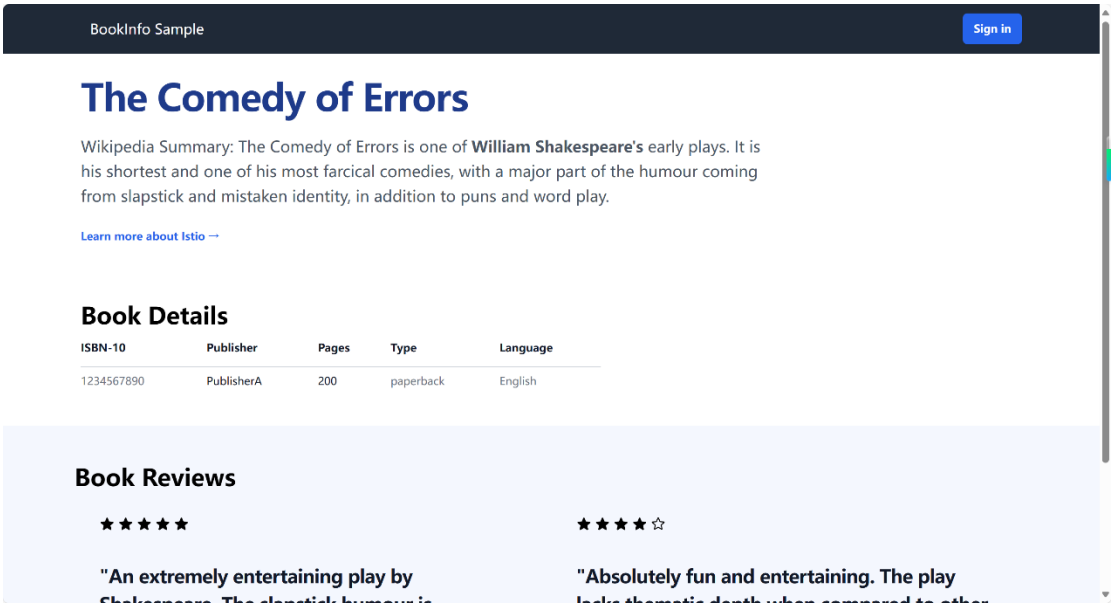
Test if the app is working inside the cluster:

```
u6237@Yezo:~/istio-1.29.2$ kubectl exec "$(kubectl get pod -l app=ratings -o jsonpath='{.items[0].metadata.name}')" -c ratings -- curl -sS productpa
ge:9080/productpage | grep -o "<title>.*</title>"
<title>Simple Bookstore App</title>
```

3.6 Expose the Application via Gateway

```
u6237@Yezo:~/istio-1.29.2$ kubectl apply -f samples/bookinfo/gateway-api/bookinfo-gateway.yaml
gateway.gateway.networking.k8s.io/bookinfo-gateway created
httproute.gateway.networking.k8s.io/bookinfo created
```

```
u6237@Yezo:~/istio-1.29.2$ kubectl annotate gateway bookinfo-gateway networking.istio.io/service-type=ClusterIP --namespace=default
gateway.gateway.networking.k8s.io/bookinfo-gateway annotated
u6237@Yezo:~/istio-1.29.2$ kubectl get gateway
NAME               CLASS   ADDRESS                                          PROGRAMMED   AGE
bookinfo-gateway   istio   bookinfo-gateway-istio.default.svc.cluster.local   True         32s
```



3.7 Access Observability Dashboards

Istio provides several dashboards for monitoring and tracing.

Install Kiali and other addons

```
Handling connection for 8080
^Cu6237@Yezo:~/istio-1.29.2kubectl apply -f samples/addons/kiali.yamlml
serviceaccount/kiali created
configmap/kiali created
clusterrole.rbac.authorization.k8s.io/kiali created
clusterrolebinding.rbac.authorization.k8s.io/kiali created
service/kiali created
deployment.apps/kiali created
u6237@Yezo:~/istio-1.29.2$ kubectl rollout status deployment/kiali -n istio-system
Waiting for deployment "kiali" rollout to finish: 0 of 1 updated replicas are available...
deployment "kiali" successfully rolled out
```

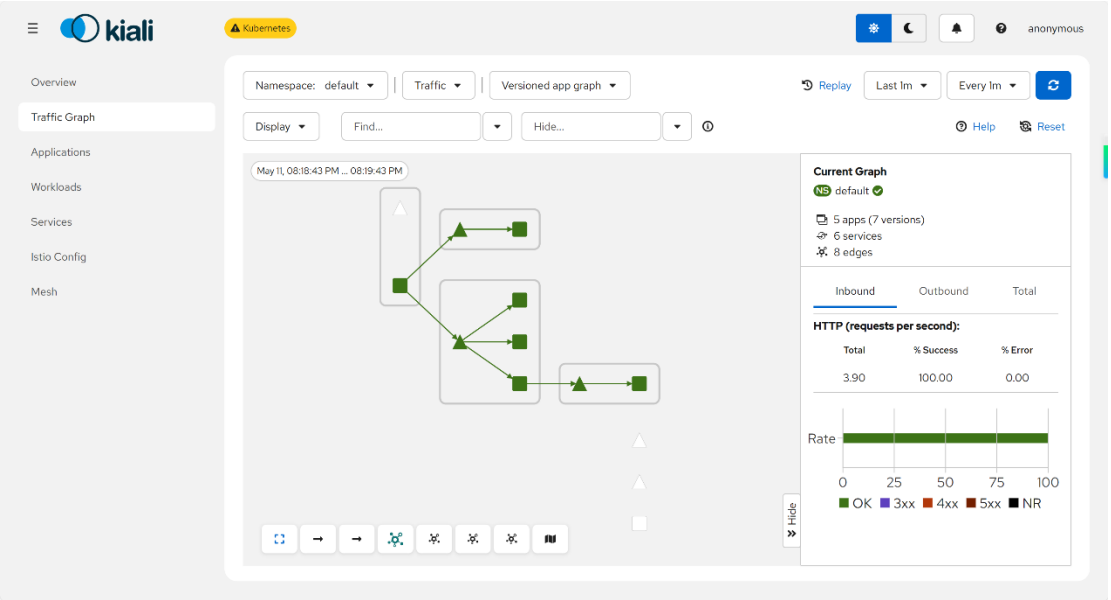
Access Kiali Dashboard

Run this script to simulate user traffic:

```
for i in {1..100}; do
  curl -s -o /dev/null "http://localhost:8080/productpage"
  sleep 0.5
done
```

The dashboard shows:

- **Service Graph** - Visual representation of service dependencies
- **Traffic flow** between productpage, details, reviews, ratings
- **Health status** of all services (green checkmarks)
- **Request volume** and **error rates**



4. Verification Checklist

- Istio installed with Gateway API support
- Default namespace labeled for sidecar injection
- Bookinfo application running (6 pods all Running)
- Each pod has sidecar proxy (2/2 containers)
- Gateway and HTTPRoute created
- External access working via port-forward/NodePort
- Kiali dashboard accessible

- Grafana metrics viewable
- Jaeger traces collected

5. **Troubleshooting Notes**

Issue	Solution
Pods stuck in ContainerCreating	Check resources: minikube stop && minikube start --cpus=4 --memory=8192
Gateway address <pending>	Ensure Gateway API CRDs installed first
Sidecar not injected	Verify label: kubectl label namespace default istio-injection=enabled --overwrite
Port-forward disconnects	Use nohup or tmux for long-running port-forward

6. **Conclusion**

This lab successfully demonstrates:

1. **Automatic sidecar injection** – Each pod automatically receives an Envoy proxy
2. **Gateway API** – Modern, role-oriented ingress configuration
3. **Traffic routing** – External user requests reach the productpage service
4. **Observability stack** – Kiali provides service graph view

The Istio service mesh provides powerful capabilities for traffic management, security, and observability with minimal application code changes.